

Plano de Implementação — Liv Pub × Vexo

Plano modular para execução por **2 devs em paralelo** sem bloqueio mútuo. Base: `Contrato_Vexo_LivPub.docx` (escopo fechado, Cl. 2.2) + deck `index.html` . Mapa de origem: ver seção "Mapa atual" abaixo e `CONTRACT.md` . **PRÉ-REQUISITO** (✅ núcleo feito 2026-06-21): segmentação unificada — ver `docs/PLANO-SEGMENTACAO.md` . Catálogo dinâmico por empresa (`segmentation_config` v2: `fields[]` + `featuredKpis`), shape único `{filters: [{field,operator,value}]}` , matcher único no backend. `perfil_musical` da Liv agora é só uma entrada no catálogo — zero motor de filtro novo.

0. Escopo contratual (canônico — taxativo)

- Base: **21.000** contatos (contrato), não 16k (deck é comercial).
- Setup: **5 semanas, R\$ 25.000** (5× R\$ 5.000 semanais).
- Operação: **R\$ 2.500/mês**, inicia 30 dias após a 5ª parcela.
- Entregas Fase 1 (Cl. 3.1): WhatsApp como central; segmentação da base; **5 esteiras**; IA de atendimento; treinamento; primeiras campanhas.
- **5 esteiras (Anexo I)**: (1) Pré-venda evento · (2) Camarote/VIP · (3) Aniversariante · (4) Reativação · (5) Pós-evento.
- **Sem garantia de resultado** (Cl. 2.3 / 10ª) — calculadora e projeções do deck NÃO viram feature.
- **Zig/Sympla** (Cl. 3.2): best-effort, progressivo, **vira adicional** (Cl. 5ª) se depender de API/credencial de terceiro. **Não entra no caminho crítico do Setup**.

1. Mapa atual (o que já existe)

Capacidade	Onde	Estado
WhatsApp central (Evolution)	<code>Conexoes.tsx</code> , <code>EvolutionAdmin.tsx</code> , <code>lead_client_evolution_instances</code> , <code>WhatsAppInbox.tsx</code>	✅
IA atendimento/qualificação	<code>chatbot-ai-engine.js</code> , <code>hardcoded-chatbot*.js</code> , <code>ChatbotConfig/Templates/Kanban</code> , <code>PromptEditor.tsx</code>	✅
Disparo + anti-ban + cota/chip	<code>campaign-outbound.js</code> , <code>campaign-ai.js</code>	✅
Import + segmentação base	<code>LeadImports.tsx</code> + segmentação unificada (<code>segmentation.js</code> , catálogo <code>fields[]</code>)	✅ dinâmica; <code>perfil_musical</code> = entrada no catálogo
Relatório / métricas	<code>Relatorios.tsx</code> , <code>commercial-intelligence.js</code>	✅
Suporte	<code>HelpDeskWidget.tsx</code> , <code>helpdesk-ai.js</code>	✅

Capacidade	Onde	Estado
Motor de régua (follow-up)	<code>followup/</code> (cron 6h, BullMQ worker, templates)	🟡 só gatilhos <code>on_schedule</code> / <code>before_meeting</code> / <code>after_meeting</code> / <code>no_reply</code>
Hospedagem	Easypanel (<code>bk-vexo</code> , <code>db-vexo</code> , Redis)	✅

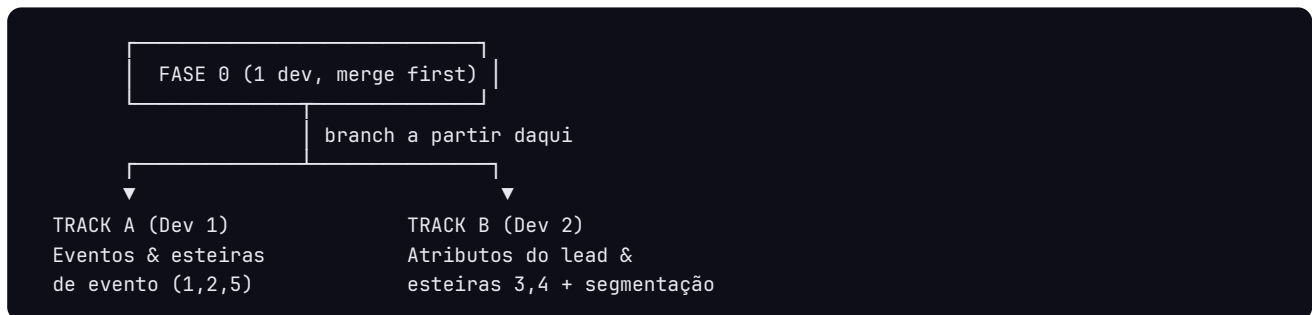
Ausências que bloqueiam as esteiras (confirmado por grep — zero ocorrências):

- Entidade **Evento** (data do evento) → bloqueia esteiras 1, 2, 5.
- Campo `data_nascimento` → bloqueia esteira 3.
- Campo `ultima_visita` / `presença` → bloqueia esteira 4 real.
- Link de pagamento/checkout (`pay.livpub/...` no deck) → esteiras 1 e 2.

2. Princípio de paralelização

Dois devs nunca devem editar o mesmo arquivo na mesma janela. Garantido por:

- Fase 0 (foundation)** — UM dev, ~meio dia, **merge primeiro**. Cria os pontos de extensão compartilhados para que os tracks não toquem os mesmos arquivos depois.
- Split por entidade, não por camada**. Cada track é vertical (migration → backend → frontend) e dono exclusivo dos seus arquivos novos.
- Migrations timestampadas separadas** — nunca colidem.
- Registry pattern no motor de follow-up** — cada track registra seu handler em arquivo próprio; ninguém edita o `switch`.



3. FASE 0 — Foundation (1 dev, merge antes de abrir os tracks)

Objetivo: criar extensões compartilhadas. Curto e auto-contido.

0.0 — Segmentação unificada (✅ feito, pré-requisito) — `docs/PLANO-SEGMENTACAO.md`. Já mergeável: catálogo `fields[]`, shape `filters[]`, matcher único, endpoint preview. Tracks já contam com isso.

0.1 — Trigger registry no follow-up

- Refatorar `calcScheduledFor` (`backend/src/followup/service.js:54`) de `switch` para um mapa `triggerHandlers = { tipo: (ctx) => Date|null }`.
- Handlers atuais migram 1:1 (sem mudança de comportamento).
- Expor `registerTriggerHandler(type, fn)` para os tracks plugarem novos tipos em arquivos próprios (`followup/triggers/birthday.js` , `followup/triggers/event.js` etc.).

- Critério de aceite: campanhas de follow-up existentes seguem funcionando idêntico (rodar smoke test atual + colar saída).

0.2 — Interface de link de pagamento (contract-first stub)

- Criar `backend/src/payments/paymentLink.js` com `buildPaymentLink({ clientId, kind, ref })` retornando placeholder configurável por env/coluna do tenant.
- Sem integração real ainda — só a interface, para os dois tracks codarem contra ela sem esperar.

0.3 — Placeholders de rota/sidebar

- Adicionar em `App.tsx` + `AppSidebar.tsx` as 2 entradas novas (`/crm/eventos` , `/crm/relacionamento`) apontando para páginas stub.
- Evita que ambos editem `App.tsx` / `AppSidebar.tsx` depois (único ponto de colisão de UI). Cada track só preenche sua página.

Entregável Fase 0: 1 PR pequeno, mergeado na `main` antes de qualquer track começar.

4. TRACK A (Dev 1) — Eventos & esteiras de evento

Dono de: esteiras **1 (Pré-venda)**, **2 (Camarote/VIP)**, **5 (Pós-evento)**.

A1 — Entidade Evento

- Migration nova: `events` (`id` , `client_id` , `name` , `event_at` , `created_at` , ...). **Multi-tenant:** `client_id` obrigatório **em toda query** (CLAUDE.md §8).
- CRUD backend em domínio próprio (`backend/src/domains/events/` ou módulo isolado). Não tocar `server.js` monolítico além do registro de rota.
- Reaproveita o anchor de data: esteiras pré/pós usam os gatilhos `before_meeting` / `after_meeting` já existentes, passando `event_at` como anchor (mínima mudança no engine).

A2 — Esteira 1: Pré-venda de evento

- Campanha segmentada disparada N dias antes do `event_at` (gatilho `before_meeting`).
- Convite + `buildPaymentLink(kind="ingresso")` (interface da Fase 0).

A3 — Esteira 2: Camarote/VIP

- Trilha dedicada com handoff p/ IA assistida (reusa `chatbot-ai-engine.js` , prompt específico).
- `buildPaymentLink(kind="camarote")` .

A4 — Esteira 5: Pós-evento

- Gatilho `after_meeting` a partir de `event_at` : agradecimento + cupom de retorno.

A5 — Frontend `Eventos.tsx`

- Preenche a página stub criada na Fase 0. CRUD de evento + status das 3 esteiras.

Arquivos exclusivos do Track A: migration `*_events.sql` , `backend/src/domains/events/*` , `backend/src/followup/triggers/event.js` (se precisar de handler novo), `frontend/src/pages/Eventos.tsx` .

5. TRACK B (Dev 2) — Atributos do lead & esteiras 3/4 + segmentação

Dono de: esteiras **3 (Aniversário)**, **4 (Reativação)** + segmentação por perfil.

B1 — Colunas no lead

- Migration nova: `ADD COLUMN IF NOT EXISTS data_nascimento DATE`, `ultima_visita DATE`, `perfil_musical TEXT` (ou tabela de tags) na tabela de leads do tenant.
- Idempotente (segue padrão `ALTER TABLE ... IF NOT EXISTS` já usado em `chatbot-ai-engine.js:71`).

B2 — Esteira 3: Aniversariante

- Novo trigger `birthday` registrado via `registerTriggerHandler` (Fase 0) em `followup/triggers/birthday.js`.
- Cron diário varre `data_nascimento` do mês/dia → cria sugestão (não envia automático; segue padrão atual do `automationEngine` = sugestão p/ aprovação).
- Oferta + montagem de lista.

B3 — Esteira 4: Reativação por inatividade

- Novo trigger `inactivity` em `followup/triggers/inactivity.js`: leads com `ultima_visita < NOW() - X`.
- Distinto do `no_reply` atual (que só olha resposta). Reusa métrica de `commercial-intelligence.js:314`.

B4 — Segmentação por perfil (CONFIG, não código) desbloqueado

- Segmentação já unificada e dinâmica (`docs/PLANO-SEGMENTACAO.md` núcleo feito).
- Só adicionar o campo `perfil_musical` ao **catálogo** (`segmentation_config.fields[]`) da empresa Liv Pub — via `PATCH /api/lead-clients/:tenantId/segmentation-config` ou modelo `balada` em `buildDefaultSegmentationConfig ( server.js )`.
- Filtro de disparo já funciona via shape `{filters:[{field,operator,value}]}` + matcher `LeadMatchesSegmentation`. Zero motor novo.
- Preview disponível no endpoint dry-run `POST /api/lead-clients/:tenantId/segmentation/preview`.

Arquivos exclusivos do Track B: migration `*_lead_attributes.sql`, `backend/src/followup/triggers/birthday.js`, `.../inactivity.js`, `frontend/src/pages/Relacionamento.tsx`.

6. Matriz anti-colisão (quem toca o quê)

Arquivo	Fase 0	Track A	Track B
<code>followup/service.js</code> (switch → registry)		—	—
<code>payments/paymentLink.js</code>	 cria	usa	usa
<code>App.tsx</code> / <code>AppSidebar.tsx</code>	 stubs	—	—
Migrations	—	<code>*_events</code>	<code>*_lead_attributes</code>
<code>domains/events/*</code>	—		—
<code>followup/triggers/*.js</code>	—	<code>event.js</code>	<code>birthday.js</code> , <code>inactivity.js</code>
<code>pages/Eventos.tsx</code>	stub		—
<code>pages/Relacionamento.tsx</code>	stub	—	

Arquivo	Fase 0	Track A	Track B
<code>campaign-outbound.js</code> (segmentation)	—	—	

Único arquivo que ambos *importam* (não editam): `payments/paymentLink.js` e o registry. Sem edição concorrente.

7. Cronograma 5 semanas (Anexo II do contrato)

Semana	Track A	Track B	Marco contratual
1	Fase 0 (1 dev) → A1 Evento	B1 colunas (após Fase 0)	Coleta base + acessos; régua inicial
2	A2 Pré-venda	B4 catálogo <code>perfil_musical</code> (config) + B3 início	WhatsApp central; segmentação 21k
3	A3 Camarote + A4 Pós-evento	B2 Aniversário + B3 Reativação	5 esteiras + IA
4	A5 frontend Eventos	B4 UI + ajustes	Integrações progressivas (Zig/Sympla se viável); treinamento
5	Ajustes finais, validação	Ajustes finais, validação	Conclusão do Setup

Prazo só corre com insumos do cliente (Cl. 6.2): base, acessos, números WhatsApp, aprovações.

8. Critérios de aceite por módulo (evidência real — CLAUDE.md §3)

Cada módulo só "pronto" com: migration aplicada (saída colada) + request real (curl/log) + filtro `client_id` provado + teste manual descrito. Sem "funciona" sem prova.

9. Fora de escopo (Cl. 5ª — só com aditivo)

Zig/Sympla (se exigir API paga de terceiro), identidade visual, tráfego pago, conteúdo, esteiras além das 5, novos módulos, checkout real (a interface da Fase 0 é stub; gateway real = adicional).